

Geradores em Python

Mário Leite

O que é um Gerador?

Um gerador é uma função especial que, em vez de retornar todos os valores de uma vez, os entrega um por um, sob demanda. Para isso, usa a palavra-chave **yield** no lugar de **return**. A cada chamada de **next()**, o gerador retorna o próximo valor e pausa a execução — preservando o estado interno — até ser chamado novamente.

Por que usar Geradores?

- Consomem muito menos memória: os valores são gerados conforme necessário, não todos de uma vez.
- Ideais para sequências longas ou infinitas: contadores, séries matemáticas, fluxos de dados.
- Código mais limpo e expressivo que loops manuais com listas auxiliares.

Exemplo prático 1 - Gerador simples em loop

O programa **UsandoGeradores1** mostra um gerador que produz números de **0** até **N**:

```
def GeraNumeros(n):           # função geradora
    for i in range(1, n+1):
        yield i               # entrega um valor e pausa

# Usando o gerador
for num in GeraNumeros(5):
    print(num)                # imprime 0, 2, 3, 4, 5
```

Saída do programa: **1 2 3 4 5** - cada valor gerado apenas quando o *loop* pede.

Exemplo prático 2 - Gerador aplicado a investimentos

O programa **UsandoGeradores2** usa um gerador para simular o crescimento de um investimento ao longo de 10 anos:

```
def SimulaInvestimento(valor_ini, taxa): # gerador financeiro
    saldo = valor_ini
    while True:
        yield round(saldo, 2)           # entrega o saldo atual
        saldo *= (1 + taxa)             # aplica juros e pausa

# Usando o gerador por 10 anos
gen = SimulaInvestimento(1000.0, 0.05) # R$ 1.000 a 5% a.a.
for ano in range(1, 11):
    print(f'Ano {ano}: R$ {next(gen)}')
```

Quando usar na prática?

- Leitura linha a linha de arquivos grandes (sem carregar tudo na memória).
- Geração de sequências matemáticas: Fibonacci, primos, séries financeiras.
- Pipelines de processamento de dados em etapas encadeadas.
- Simulações que precisam de estado entre chamadas.

Em Python, **yield** transforma qualquer função em um gerador - entregando valores um a um, preservando o estado entre chamadas e economizando memória; simples, elegante e poderoso. Veja os dois programas abaixo.

```
'''
UsandoGeradores1.py
-----

Geradores são funções especiais em Python que permitem a geração de
sequências de valores de forma eficiente sob demanda, definidos com a
palavra-chave yield. Este programa simples mostra como implementar um
gerador em um loop.
-----

'''

def GerarPares(limite):
    i = 0
    while(i <= limite):
        yield i
        i += 2

=====
#Programa principal
limite = 10 #define o limite do gerador
gerador = GerarPares(limite)
for numero in gerador:
    print(numero)
#Fim do programa "UsandoGeradores" -----
```

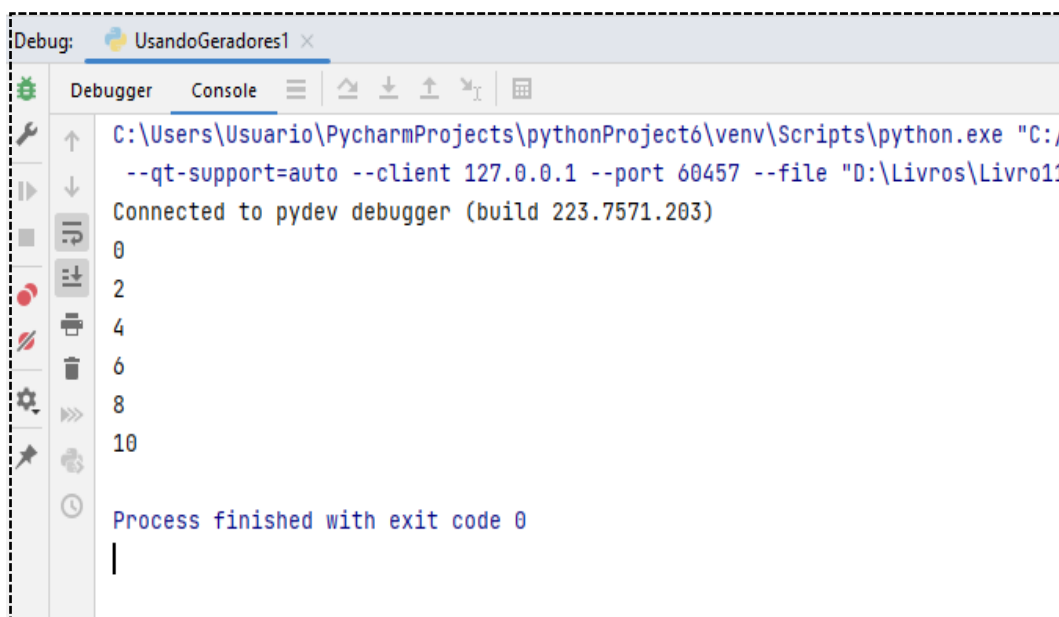


Figura 1- Saída do programa “UsandoGeradores1”

```
'''
UsandoGeradores2.py
-----

Exemplo simples para demonstrar o funcionamento de um Gerador aplicado
a um sistema de investimento com taxa de juros e investimento inicial em
dez anos.
-----

'''

def CalcularJuros(investimento, taxa, anos):
    saldo = investimento
    for ano in range(1, anos + 1):
        saldo *= 1 + taxa
        yield ano, saldo

#=====
#Programa principal
#Definie o investimento inicial
invest = 10000

#Define a taxa de juros anual (5%)
taxaJuros = 0.05

#Define o número de anos da aplicação
anos = 10

#Chama a função para iterar sobre o gerador
for ano, saldo in CalcularJuros(invest, taxaJuros, anos):
    print(f'Ano {ano}: Saldo: R${saldo:.2f}')
#Fim do programa "UsandoGeradores2" -----
```

```
Run: UsandoGeradores2 x
C:\Users\Usuario\PycharmProjects\pythonProject6\venv\Scripts\python.exe "D:\Livros\Livro11"
Fim do ano 1: Saldo: R$10500.00
Fim do ano 2: Saldo: R$11025.00
Fim do ano 3: Saldo: R$11576.25
Fim do ano 4: Saldo: R$12155.06
Fim do ano 5: Saldo: R$12762.82
Fim do ano 6: Saldo: R$13400.96
Fim do ano 7: Saldo: R$14071.00
Fim do ano 8: Saldo: R$14774.55
Fim do ano 9: Saldo: R$15513.28
Fim do ano 10: Saldo: R$16288.95

Process finished with exit code 0
```

Figura 2 - Saída do programa “UsandoGeradores2”